



UNIVERSIDAD TECNOLÓGICA NACIONAL

Facultad Regional Mendoza

Ingeniería en Sistemas de Información

Guía Práctica N.º 5

Sistema de Entrada/Salida

Cátedra: **Sistemas Operativos** • Unidad Temática N.º 5

Trabajo grupal — 4 a 5 integrantes

Docentes: Ing. Martínez • Lic. Faccio • Ing. Marques • Ing. Ortiz • Ing. Pennisi • Ing. Edera

Ciclo lectivo 2026

Consigna general

Responder la siguiente actividad que introduce conceptos básicos de los mecanismos de Entrada/Salida. Consulte la bibliografía recomendada en el cronograma de actividades prácticas.

Propósitos de la práctica

- Resolver problemas relacionados con el hardware de entrada/salida.
- Resolver problemas relacionados con el software de entrada/salida.
- Utilizar comandos para analizar la entrada y salida en distintos sistemas operativos.

Tiempo de ejecución

Comenzará con la actividad durante la clase de práctica y en caso de ser necesario, continúe con ella extra clase. Recuerde que debe presentar la actividad práctica la semana próxima, de manera digital o impresa. Tenga en cuenta las recomendaciones para la presentación de trabajos prácticos que se sugieren al comienzo de este cuadernillo.

Carácter de la actividad: Grupal

Evidencias de logro

- Distingue las características más relevantes de los dispositivos de entrada salida y controladores de dispositivos.
- Evalúa y resuelve satisfactoriamente problemas relacionados con accesos a dispositivos de entrada salida.
- Compara y analiza los mecanismos de entrada y salida en distintos sistemas operativos.
- Realiza un informe escrito satisfactorio, en tanto:
 - Respeta plazos de entrega de la actividad práctica.
 - Desarrolla los temas con pertinencia y precisión conceptual.
 - Respeta criterios formales de presentación del informe.
- Autorregula la propia conducta en el cumplimiento de compromisos acordados.

Parte A — Ejercicios Obligatorios Sincrónicos

1. Utilizar el emulador 8086. Almacenar en memoria una cadena de caracteres y mostrarla en pantalla (carácter por carácter), usando la **INT 10H**. A continuación se muestra el programa propuesto.

Documente la traza del programa línea por línea. ¿Cuál es la salida estándar de ejecución de este programa?

```
.MODEL SMALL
.STACK 64
.DATA
MSG DB 'Bienvenidos a la materia Sistemas Operativos'
.CODE
MAIN PROC FAR
MOV AX,@DATA
MOV DS,AX
MOV CX,2CH          ; n° de caracteres a mostrar
LEA SI,MSG           ; SI -> primer carácter
MOV BH,00H          ; página de video 0
TI: MOV AL,[SI]      ; AL = carácter actual
MOV AH,0EH          ; func 0EH de INT 10H (teletipo)
INT 10H             ; muestra AL y avanza el cursor
INC SI              ; siguiente carácter
```

```

        DEC CX
        JNZ TI          ; repetir hasta CX = 0
        MOV AX,4C00H    ; salida a DOS (func 4CH)
        INT 21H
MAIN    ENDP
        END MAIN

```

2. Utilizar el emulador 8086. Usando la **INT 10H**, mostrar en pantalla una cadena de caracteres almacenada en memoria. Los caracteres se deben mostrar todos juntos.

- Comente cada instrucción indicando su propósito.
- Verifique que el programa funciona en forma correcta, y agregue al final del ejercicio la captura de pantalla correspondiente.

```

        .MODEL SMALL
        .STACK 64
        .DATA
MSG      DB 'Bienvenidos a la materia Sistemas Operativos'
        .CODE
MAIN     PROC FAR
        MOV AX,@DATA
        MOV DS,AX
        MOV ES,AX
        LEA BP,MSG
        MOV AH,13H
        MOV AL,00H
        MOV BH,00H
        MOV BL,1EH
        MOV CX,2CH
        MOV DX,0300H
        INT 10H
        MOV AX,4C00H
        INT 21H
MAIN     ENDP
        END MAIN

```

3. Utilizar el emulador 8086. Usando la **INT 21H** mostrar en pantalla los caracteres que son ingresados en el teclado.

- Escriba el código fuente del programa y coméntelo instrucción por instrucción.
- Responda: **¿Cuál es el carácter que debe introducirse para que el programa finalice?**

4. Utilizar el emulador 8086. Usando la **INT 21H** mostrar en pantalla el string: "Ejercicio Muestra esta cadena de caracteres\$". **¿Cuál es el propósito del carácter \$ en el string a mostrar?**

```

        .MODEL SMALL
        .STACK 64
        .DATA
MSG      DB 'Ejercicio Muestra esta cadena de caracteres$'
        .CODE
MAIN     PROC FAR
        MOV AX,@DATA
        MOV DS,AX
        LEA DX,MSG          ; DX -> cadena (func 09H)
        MOV AH,09H         ; func 09H: mostrar cadena terminada en '$'

```

```

        INT    21H
        MOV    AX,4C00H        ; salida a DOS
        INT    21H
MAIN     ENDP
        END    MAIN

```

5. Utilizar el emulador 8086. Usando la **INT 21H** ingresar un string por teclado y mostrarlo en pantalla. Para la resolución escriba el código fuente del programa y coméntelo.

6. Utilizar el emulador 8086. Usando la **INT 16H** lea un carácter desde el teclado y muéstrela en pantalla usando la **INT 21H**. Repita la operación hasta tanto se presione la tecla **ESC**. Para la resolución escriba el código fuente del programa y coméntelo.

7. La UART.

a) **Investigue qué es la UART.**

b) **Analice el siguiente programa**, el cual usa las especificaciones técnicas de la UART.

Puertos disponibles (Port Address / Interrupt)

Puerto	Dirección	IRQ
COM1	3F8	IRQ4
COM2	2F8	IRQ3
COM3	3E8	IRQ4*
COM4	2E8	IRQ3*

Registros de la UART

Offset	Nombre	Función	Uso
0*	DATA	Data Read/Write Reg.	Line I/O
1*	IER	Interrupt Enable Reg.	Enable Tx, Rx, RxError, Modem int.
2	IID	Interrupt ID Reg.	ID of highest interrupt source.
3	LCR	Line Control Reg.	Line control parameters and Break
4	MCR	Modem Control Reg.	DTR, RTS, OUT1, OUT2 and loopback.
5	LSR	Line Status Reg.	Tx and Rx status (PE, FE, OE)
6	MSR	Modem Status Reg.	CTS, DSR, RI, RLSD & changes.
0*	DLL	Divisor Latch LOW	LSB of Baudrate divisor
1*	DLH	Divisor Latch HIGH	MSB of Baudrate divisor

(*) Nota: el bit 7 del LCR controla el acceso a DATA/IER (=0) o a los Divisor Latches (=1).

Tabla de velocidades (BPS) — divisor de baudios

Velocidad (BPS)	Divisor (dec)	DLH (byte alto)	DLL (byte bajo)
50	2304	09H	00H
300	384	01H	80H
600	192	00H	C0H
1200	96	00H	60H
2400	48	00H	30H
4800	24	00H	18H
9600	12	00H	0CH

Velocidad (BPS)	Divisor (dec)	DLH (byte alto)	DLL (byte bajo)
19200	6	00H	06H
38400	3	00H	03H
57600	2	00H	02H
115200	1	00H	01H

LCR — Registro de Control de Línea (offset 3)

Bit(s)	Función
7	DLAB: 1 = acceder a DLL/DLH (offsets 0 y 1); 0 = acceder a DATA/IER
6	Send Break: envía SPACE hasta que el bit se vuelve a poner en 0
5-3	Control de paridad: XX0 = sin paridad; 001 = impar; 011 = par; 101 = stick TRUE (Mark); 111 = stick FALSE (Space)
2	Bits de stop: 0 = 1 stop; 1 = 1.5 (si la palabra es de 5 bits) o 2 stop (resto)
1-0	Longitud de palabra: 11 = 8 bits; 10 = 7; 01 = 6; 00 = 5

MCR — Registro de Control de Módem (offset 4)

Bit	Señal
4	Modo loopback (la UART se prueba a sí misma)
3	OUT2
2	OUT1
1	RTS
0	DTR

IER — Registro de Habilitación de Interrupciones (offset 1)

Bit	Propósito	Causa de la interrupción
0	Rx Data Ready	Se recibió un carácter
1	THRE	Hay lugar para enviar otro carácter
2	Rx status	PE, FE, overrun y/o BREAK
3	Modem status	Cambio en CTS, DSR, RI o DCD/RLSD
4-7	(no se usan)	—

LSR — Registro de Estado de Línea (offset 5)

Bit	Nombre	Significado
6	TEMT (TSE) — Tx Shift Reg. Empty	Se terminó de enviar un carácter
5	THRE — Tx Holding Register Empty	Se puede escribir un carácter para enviar
4	Break received	Se recibió un BREAK
3	FE — Framing Error	Cantidad de bits de datos incorrecta
2	PE — Parity Error	Paridad incorrecta
1	OE — Overrun Error	Llegó un dato antes de leer el anterior
0	Rx Data Ready	Hay un carácter disponible para leer en el RBR

Qué hace el programa. Trabaja sobre COM1 (base 3F8H). Primero configura la UART y después entra en un bucle de **sondeo (polling)** del registro de estado:

- a) **DLAB = 1** (escribe 80H en el LCR) para poder cargar el divisor de baudios.
- b) **Divisor = 0060H** (60H en DLL, 00H en DLH) → **1200 BPS** (ver tabla de velocidades).
- c) **DLAB = 0 y formato de línea** (0AH en el LCR) → 7 bits de datos, paridad impar, 1 bit de stop.
- d) **MCR = 13H** → activa DTR, RTS y **modo loopback** (la UART se conecta su salida a su entrada, por eso el programa puede probarse a sí mismo).
- e) **IER = 00H** → no usa interrupciones; trabaja por sondeo.

En el bucle lee el **LSR** y decide: si hay error de línea va a **ERROR**; si hay un carácter recibido va a **RECIBIR**; si el transmisor está libre, toma una tecla del teclado y la envía. Con la tecla **ESC** termina.

Las constantes EQU del encabezado le ponen nombre a cada puerto y a cada máscara, de modo que el código se lee mapeándolo directamente contra las tablas de registros de arriba (en lugar de números mágicos).

```
.MODEL TINY
.CODE
ORG 100H

; ---- Constantes de la UART (COM1, base 3F8H) ----
COM1    EQU 3F8H
THR     EQU COM1      ; Transmitter Holding Reg (Tx) - offset 0
RBR     EQU COM1      ; Receiver Buffer Reg (Rx) - offset 0
IER     EQU COM1+1    ; Interrupt Enable - offset 1
LCR     EQU COM1+3    ; Line Control - offset 3 (3FBH)
MCR     EQU COM1+4    ; Modem Control - offset 4 (3FCH)
LSR     EQU COM1+5    ; Line Status - offset 5 (3FDH)
DLL     EQU COM1      ; Divisor Latch Low (con DLAB=1)
DLH     EQU COM1+1    ; Divisor Latch High (con DLAB=1)
ERR_MASK EQU 1EH      ; LSR bits 1-4: OE/PE/FE/Break
RX_RDY  EQU 01H       ; LSR bit 0: hay dato recibido
TX_RDY  EQU 20H       ; LSR bit 5: THRE, transmisor libre

MAIN    PROC    NEAR
.STARTUP

; ---- Configuración de la UART ----
MOV DX,LCR
MOV AL,80H      ; DLAB=1: habilita el acceso a los divisor latches
OUT DX,AL
MOV DX,DLL
MOV AL,60H      ; divisor 96 -> 1200 BPS (ver tabla de velocidades)
OUT DX,AL
MOV DX,DLH
MOV AL,00H
OUT DX,AL
MOV DX,LCR
MOV AL,0AH      ; DLAB=0; 7 bits de datos, paridad impar, 1 stop
OUT DX,AL
MOV DX,MCR
MOV AL,13H      ; DTR + RTS + loopback (la UART se prueba a si misma)
OUT DX,AL
MOV DX,IER
MOV AL,00H      ; sin interrupciones: se trabaja por sondeo (polling)
OUT DX,AL

; ---- Bucle principal: sondea el estado de la línea (LSR) ----
PORSIEMPRE:
```

```

MOV DX,LSR
IN AL,DX
TEST AL,ERR_MASK ; hay algun error de linea (OE/PE/FE/Break)?
JNZ ERROR
TEST AL,RX_RDY ; hay caracter recibido?
JNZ RECIBIR
TEST AL,TX_RDY ; el transmisor esta libre (THRE)?
JZ PORSIEMPRE
MOV AH,1 ; hay tecla en el buffer del teclado?
INT 16H
JZ PORSIEMPRE
MOV AH,0 ; leer la tecla
INT 16H
CMP AH,01H ; scan code 01H = ESC -> terminar
JZ FIN
MOV DX,THR ; transmitir el caracter tipeado
OUT DX,AL
JMP PORSIEMPRE

; ---- Caracter recibido: leerlo del RBR y mostrarlo ----
RECIBIR:
MOV DX,RBR
IN AL,DX
AND AL,7FH ; descartar el bit de paridad (7 bits de datos)
PUSH AX
MOV BX,0
MOV AH,0EH ; teletype (INT 10H): mostrar caracter
INT 10H
POP AX
CMP AL,0DH ; fue Enter (CR)?
JNZ PORSIEMPRE
MOV AL,0AH ; si: agregar salto de linea (LF)
MOV BX,00
MOV AH,0EH
INT 10H
JMP PORSIEMPRE

; ---- Manejo de error de linea ----
ERROR:
MOV DX,RBR
IN AL,DX ; leer el dato para limpiar la condicion
MOV AL,0A8H ; caracter de marca de error a mostrar
MOV BX,0
MOV AH,0EH
INT 10H
JMP PORSIEMPRE

FIN: .EXIT
MAIN ENDP
END

```

Parte B — Ejercicios Obligatorios Asincrónicos

1. Identificación de dispositivos de E/S.

Objetivo: Identificar y comprender los distintos dispositivos de entrada/salida en el sistema.

- a) **Listar dispositivos de bloque.** Utiliza `lsblk` para listar los dispositivos de bloque. *Pregunta: ¿Qué información proporciona `lsblk`?*

- b) **Listar dispositivos de caracteres.** Utiliza `ls -l /dev`. *Pregunta: ¿Cómo puedes identificar un dispositivo de caracteres en la salida de `ls`?*

```
ls -l /dev | grep '^c'
```

- c) **Explorar el archivo `/proc/devices`.** `cat /proc/devices`. *Pregunta: ¿Qué diferencia hay entre los dispositivos de caracteres y los de bloque?*

2. Creación de dispositivos virtuales.

Objetivo: Crear y utilizar dispositivos virtuales en Linux.

- a) **Crear un dispositivo de bucle (loop device).**

```
dd if=/dev/zero of=loopback.img bs=1M count=50
sudo losetup /dev/loop0 loopback.img
lsblk | grep loop0
```

- b) **Formatear y montar el dispositivo de bucle.**

```
sudo mkfs.ext4 /dev/loop0
sudo mkdir /mnt/loopback
sudo mount /dev/loop0 /mnt/loopback
df -h | grep loop0
```

- c) **Desmontar y liberar el dispositivo de bucle.**

```
sudo umount /mnt/loopback
sudo losetup -d /dev/loop0
lsblk | grep loop0
```

3. Uso de dispositivos de caracteres.

Objetivo: Utilizar dispositivos de caracteres como `/dev/null` y `/dev/zero`.

- a) **Redirigir salida a `/dev/null`:** `ls -l / > /dev/null`. *Pregunta: ¿Qué sucede con la salida del comando cuando se dirige a `/dev/null`?*
- b) **Leer desde `/dev/zero`:** `head -c 100 /dev/zero | hexdump -C`. *Pregunta: ¿Qué tipo de datos proporciona `/dev/zero`?*

4. Creación de dispositivos personalizados.

Objetivo: Crear y utilizar dispositivos de bloque y de caracteres personalizados.

Nota: crear el nodo con `mknod ... c 240 0` solo agrega la entrada en `/dev` con ese par mayor/minor, pero **no** asocia ningún driver. Si no hay un módulo de kernel registrado para el mayor 240, las llamadas `open/read/write` sobre el nodo fallan con `ENXIO` (“No such device”). Por eso la parte directamente resoluble de este ejercicio es **crear e inspeccionar el nodo** (tipo, mayor y minor); las operaciones de E/S requieren cargar antes el módulo de kernel correspondiente con `insmod`.

- a) **Crear un dispositivo de caracteres con `mknod`:**

```
sudo mknod /dev/mychar c 240 0
ls -l /dev/mychar
```

- b) **Crear un dispositivo de bloque con `mknod`:**


```
sudo mknod /dev/myblock b 241 0
ls -l /dev/myblock
```

- c) **Inspeccionar los nodos creados.** Esta es la parte resoluble sin un driver: confirmar que el nodo existe y leer su tipo, mayor y menor.

```
ls -l /dev/mychar /dev/myblock
stat /dev/mychar
```

Pregunta: en la salida de `ls -l`, ¿qué letra inicial distingue un dispositivo de caracteres de uno de bloque?, ¿qué representan los dos números (mayor y menor) que aparecen en lugar del tamaño?

- d) **Intentar utilizar el dispositivo de caracteres personalizado:** escribe un programa en C que intente leer y escribir en el dispositivo; guardar como `char_device.c`.

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
int main() {
    int fd = open("/dev/mychar", O_RDWR);
    if (fd == -1) {
        perror("Error al abrir el dispositivo");
        return 1;
    }
    char buffer[100];
    ssize_t bytes_read = read(fd, buffer, sizeof(buffer) - 1);
    if (bytes_read == -1) {
        perror("Error al leer del dispositivo");
    } else {
        buffer[bytes_read] = '\0'; // terminar la cadena antes de imprimir
        printf("Leído del dispositivo: %s\n", buffer);
    }
    const char *message = "Hola, dispositivo de caracteres!";
    ssize_t bytes_written = write(fd, message, strlen(message));
    if (bytes_written == -1) {
        perror("Error al escribir en el dispositivo");
    }
    close(fd);
    return 0;
}
```

- e) **Compilar y ejecutar:**

```
gcc -o char_device char_device.c
sudo ./char_device
```

*Pregunta: con el nodo creado por `mknod` pero sin un módulo de kernel asociado al mayor 240, ¿cuál es la **primera** llamada de la API que falla (`open`) y qué error reporta (`ENXIO`)? ¿Por qué las llamadas `read/write` posteriores nunca llegan a ejecutarse? (ver la nota anterior sobre `ENXIO`). ¿Qué haría falta para que las operaciones de E/S funcionaran realmente?*

5. Monitoreo de la actividad de dispositivos de E/S.

Objetivo: Utilizar herramientas para monitorear la actividad de los dispositivos de E/S.

- a) **Usar `iostat`:**

```
sudo apt-get install -y sysstat  
iostat -d 1 5
```

Pregunta: ¿Qué tipo de información proporciona iostat?

b) Usar iotop:

```
sudo apt-get install -y iotop  
sudo iotop
```

Pregunta: ¿Qué tipo de información proporciona iotop?

Reflexión final

Después de completar los ejercicios, escribe una breve reflexión sobre:

- Las diferencias entre los dispositivos de bloque y de caracteres.
- Las herramientas que encontraste más útiles para gestionar y monitorear dispositivos de E/S.
- Cómo la comprensión de la gestión de dispositivos de E/S te ayuda a administrar mejor un sistema Linux.